



GCD

1 Instructions

1. Use **Behavioural** and **Structural** modeling for writing VHDL description
2. Perform RTL simulation using the provided testbench and tracefile.
3. Demonstrate the simulations to your TA
4. Perform **Scanchain** on the Xenon board and verify with your TA.
5. Submit the entire project files in .zip format in moodle.

2 Problem Statement

- Write VHDL description in **Behavioural** and **Structural** modeling to compute GCD of two 4-bit numbers A and B using repetitive subtraction.
- Reset is asynchronous in nature i.e. reset effects the output irrespective of the input clock arrival.
- On Reset, fsm should go to s_idle state.

3 Algorithm

The algorithm uses repeated subtraction and swapping, as follows:

1. Load the two numbers X and Y.
2. **Check for Completion:** Check if Y is equal to zero. If it is, the algorithm is complete and the GCD is the value of X. If not, proceed to the next step.
3. **Check for Swapping:** If X is less than Y, swap the values of X and Y so that X is always greater than or equal to Y.
4. **Perform Subtraction:** Subtract the value of Y from X and store the result back in X.
5. **Loop:** Repeat the process from step 2 until Y becomes zero.

4 How to Create a Datapath-Control Path Design in VHDL

1. **Start by designing the top-level entity, which connects both the datapath and control path modules.** The top-level module defines all input/output ports and internal signals required to interconnect the datapath and controller.
2. **Create the datapath module using structural modeling.**
 - (a) Divide the datapath into smaller components like subtractor, comparator, and multiplexers.
 - (b) Model each component separately and instantiate them in the datapath architecture using internal signals for interconnection.
 - (c) Ensure control signals such as **ld_x**, **ld_y** and **swap_en** are used to guide data movement and operations within the datapath.

3. Develop the control unit using behavioral modeling.

- (a) Use a finite state machine (FSM) approach inside a process block. **Use the FSM states described in detail in Figure 1.**
- (b) Monitor input flags from the datapath such as `y_eq_0` ($Y = 0$) and `x_lt_y` ($X < Y$), and generate appropriate control signals like `ld_x`, `ld_y` and `swap_en`.
- (c) The FSM transitions through states such as `S_IDLE`, `S_LOOP` based on input conditions.

4. Connect the datapath and controller at the top-level.

5 Control Signals and Their Roles

The controller generates specific control signals to drive the datapath:

- **ld_x:** Load enable for register X. When asserted, the datapath updates the X register with input value X.
- **ld_y:** Load enable for register Y. When asserted, the datapath updates the Y register with input value Y.
- **swap_en:** Control signal that swaps the contents of registers X and Y when $X < Y$.
- **x_lt_y** Control signal that lets you know when x is less than y.
- **y_eq_0** Control signal that lets you know when y is equal to 0.
- **ready:** Indicates completion of the GCD computation. This is asserted high when $Y = 0$.

6 Description of the states

The following FSM manages the algorithm's flow and has two states: `S_IDLE` and `S_LOOP`.

S_IDLE State

The **S_IDLE** state is the initial state where the FSM waits for a new GCD calculation to begin.

- **Entering the State:** The FSM enters this state upon a **reset** (`rst = '1'`), or after a GCD calculation is completed when the `y_eq_0` signal becomes '1' in the `S_LOOP` state.
- **Process within the State:** The FSM remains in `S_IDLE` until the **start** signal is asserted.
 - When `start = '1'`, the FSM transitions to the `S_LOOP` state.

S_LOOP State

The **S_LOOP** state is where the iterative GCD algorithm is executed. The FSM remains in this state as long as the calculation is in progress.

- **Entering the State:** The FSM transitions to this state from `S_IDLE` when `start = '1'`.
- **Process within the State:** The FSM checks the input signals `y_eq_0` and `x_lt_y` to determine the next algorithmic step.
 - **Case 1: `y_eq_0 = '1'`**
 - * This condition indicates that the GCD has been found.
 - * The **ready** signal is set to '1'.
 - * The gcd value is now sent at output.
 - * The FSM transitions back to `S_IDLE` to wait for a new calculation.
 - **Case 2: `x_lt_y = '1'` and `y_eq_0` is '0'**
 - * The value in the X register is less than the value in the Y register.

- * The **swap_en** signal is set to '1', enabling the swap operation.
- * The FSM remains in the S_LOOP state.
- **Case 3: Default Case** (x_lt_y is '0' and y_eq_0 is '0')
 - * This represents the main subtraction step of the Euclidean algorithm (where X is greater than or equal to Y).
 - * The FSM remains in the S_LOOP state to continue the iterative process.

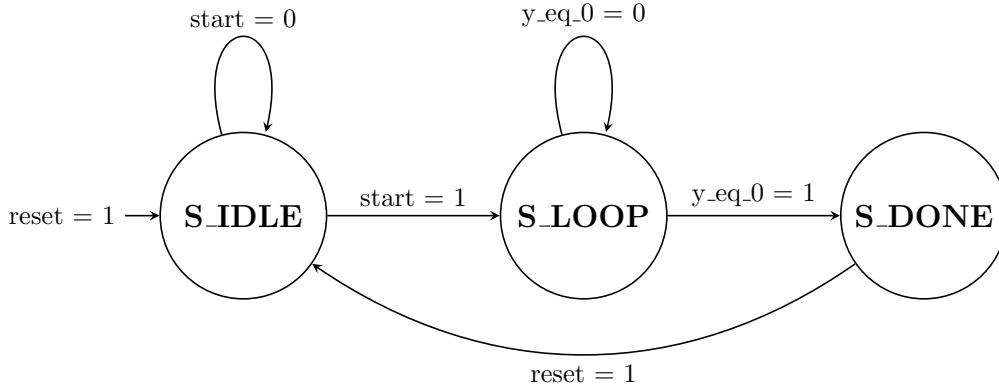


Figure 1: GCD FSM with S_DONE State

7 Design Verification:

- **Tracefile format:**
`< reset clock start A3 A2 A1 A0 B3 B2 B1 B0 > < ready Y3 Y2 Y1 Y0 > < Maskbits >`
[Tracefile](#)

Testing on Board

- Test the correctness of your design on the Xen-10 board.
- Perform the pin mapping as follows:

$A_3-A_0 \implies SW8-SW5$
 $B_3-B_0 \implies SW4-SW1$
 $Reset \implies PB1$
 $Start \implies PB2$
 $Clock \implies 1\text{ Hz On Board Clock}$
 $Y_3-Y_0 \implies LED4-LED1$
 $Ready \implies LED5$

Clock Source Frequency	FPGA Pin no.
1 Hz CLK	55
50 MHz CLK	26
Ext CLK	27
10 MHz CLK	29

Figure 2: Pin-mapping for on-board Clock Sources

Header-1			
Header pin no.	FPGA pin no.	Header pin no.	FPGA pin no.
1	75	2	76
3	77	4	78
5	79	6	81
7	84	8	85
9	86	10	87
11	88	12	89
13	90	14	91
15	92	16	93
17	96	18	98
19	99	20	100
21	101	22	102
23	105	24	106
25	110	26	111
27	113	28	114
29	118	30	119
31	GND	32	3.3V

Figure 3: Pin-mapping for Header - 1

8 Benefits of Using Structural Datapath + Behavioral Control

- Combining structural datapath and behavioral control gives you better control over how your design uses FPGA resources like LUTs.
- It also makes your design more modular and easier to understand, especially when viewed in tools like Quartus.
- You can clearly see how data moves and how decisions are made, which helps with debugging, optimizing speed, and reducing logic usage.